

# Module 5: Object-Oriented Programming (OOP) in TypeScript

## 1. Introduction to Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) in TypeScript allows developers to structure code using objects, classes, and inheritance.

## 2. Classes & Objects

A class is a blueprint for creating objects.

```
typescript
```

```
class Person {
```

```
  name: string;
```

```
  age: number;
```

```
  constructor(name: string, age: number) {
```

```
    this.name = name;
```

```
    this.age = age;
```

```
  }
```

```
  introduce(): string {
```

```
    return `Hi, I'm ${this.name} and I'm ${this.age} years old.`;
```

```
  }
```

```
}
```

```
let person1 = new Person("Alice", 25);
```

```
console.log(person1.introduce());
```

## 3. Access Modifiers (Public, Private, Protected)

### # Public (default):

```
typescript
```

```
class Car {
```

```
  public brand: string;
```

```
  constructor(brand: string) {
```

```
    this.brand = brand;
```

```
  }
```

```
}
```

## # Private (only accessible within the class):

typescript

```
class Secret {
```

```
  private password: string;
```

```
  constructor(password: string) {
```

```
    this.password = password;
```

```
  }
```

```
  reveal(): string {
```

```
    return `The password is ${this.password}`;
```

```
  }
```

```
}
```

## # Protected (accessible within the class and subclasses):

typescript

```
class Animal {
```

```
  protected species: string;
```

```
  constructor(species: string) {
```

```
    this.species = species;
```

```
  }
```

```
}
```

```
class Dog extends Animal {
```

```
  constructor() {
```

```
    super("Dog");
```

```
  }
```

```
  getSpecies(): string {
```

```
    return this.species;
```

```
  }
```

```
}
```

## 4. Inheritance

A child class inherits properties and methods from a parent class.

typescript

```
class Employee extends Person {  
  jobTitle: string;  
  
  constructor(name: string, age: number, jobTitle: string) {  
    super(name, age);  
    this.jobTitle = jobTitle;  
  }  
  
  introduce(): string {  
    return `${super.introduce()} I work as a ${this.jobTitle}.`;  
  }  
}
```

## 5. Abstract Classes

Abstract classes cannot be instantiated but can be extended.

typescript

```
abstract class Animal {  
  abstract makeSound(): void;  
}  
  
class Cat extends Animal {  
  makeSound(): void {  
    console.log("Meow!");  
  }  
}
```

```
let myCat = new Cat();  
myCat.makeSound();
```

## 6. Interfaces in OOP

Interfaces define the structure of a class.

```
typescript
```

```
interface Vehicle {  
  speed: number;  
  drive(): void;  
}
```

```
class Bike implements Vehicle {  
  speed = 50;
```

```
  drive(): void {  
    console.log(`Driving at ${this.speed} km/h`);  
  }  
}
```

## 7. Static Members

Static members belong to the class rather than instances.

```
typescript
```

```
class MathUtils {  
  static PI = 3.14;
```

```
  static circleArea(radius: number): number {  
    return this.PI * radius * radius;  
  }  
}
```

```
console.log(MathUtils.circleArea(5));
```

## Practice Questions:

1. Create a `Student` class with properties `name` and `grade`, and a method `getDetails()`.
2. Define a `BankAccount` class with a `private` balance and a method to deposit money.
3. Create an `Employee` class with `public` properties and extend it with a `Manager` class.
4. Implement an abstract class `Shape` with an abstract method `calculateArea()`.
5. Create an interface `SmartDevice` with a method `turnOn()` and implement it in a `Phone` class.

## Practice Answers:

1. `Student` class:

typescript

```
class Student {
```

```
  name: string;
```

```
  grade: number;
```

```
  constructor(name: string, grade: number) {
```

```
    this.name = name;
```

```
    this.grade = grade;
```

```
  }
```

```
  getDetails(): string {
```

```
    return `${this.name} is in grade ${this.grade}.`;
```

```
  }
```

```
}
```

2. `BankAccount` with `private` balance:

typescript

```
class BankAccount {
```

```
  private balance: number = 0;
```

```
  deposit(amount: number): void {
```

```
    this.balance += amount;
```

```
  }
```

```
  getBalance(): number {
```

```
    return this.balance;
```

```
  }
```

```
}
```

3. `Employee` and `Manager` classes:

typescript

```
class Employee {
```

```
  constructor(public name: string, public salary: number) {}
```

```
}
```

```
class Manager extends Employee {
```

```
  department: string;
```

```
constructor(name: string, salary: number, department: string) {  
  super(name, salary);  
  this.department = department;  
}  
}
```

4. `Shape` abstract class:

typescript

```
abstract class Shape {  
  abstract calculateArea(): number;  
}
```

```
class Rectangle extends Shape {  
  constructor(private width: number, private height: number) {  
    super();  
  }
```

```
    calculateArea(): number {  
      return this.width * this.height;  
    }  
  }
```

5. `SmartDevice` interface:

typescript

```
interface SmartDevice {  
  turnOn(): void;  
}
```

```
class Phone implements SmartDevice {  
  turnOn(): void {  
    console.log("Phone is now ON");  
  }  
}
```